

VETCARE

Sistema de Gestión de Veterinaria

Exposición e Implementación de Objetos Programables en Oracle

Módulo seleccionado: Citas

Integrantes del equipo:

- Ederd Carrasco
- Xiomara Garcia

Github del Proyecto: [bee-17/vetcare](https://github.com/bee-17/vetcare)

TECSUP — Curso: Base de Datos Avanzadas
Fecha: 05/07/2026

1. Descripción del sistema

VetCare es un sistema integral de gestión para clínicas veterinarias, desarrollado con Django REST Framework en el backend y React + Tailwind en el panel administrativo, sobre una base de datos PostgreSQL. El sistema automatiza los procesos operativos de una clínica veterinaria: administración de clientes y mascotas, agendamiento de citas, historial clínico, control de inventario (petshop y medicamentos categorizados) y facturación (facturas y boletas).

El sistema está organizado en cuatro módulos principales — Facturación, Historial Clínico, Citas y Doctores — más un módulo de Inventario, cada uno con sus propias operaciones de registro, actualización, eliminación y reportes especializados.

2. Descripción del módulo desarrollado: Citas

El módulo de Citas gestiona el agendamiento de consultas veterinarias entre una mascota y un veterinario disponible. Permite registrar nuevas citas, actualizar su estado a lo largo de su ciclo de vida (pendiente → confirmada → atendida, o cancelada), y generar reportes operativos como las citas del día, las citas por veterinario, las citas canceladas y las citas pendientes por mascota.

Para este trabajo se migró el diseño de este módulo a Oracle y se implementaron los siguientes objetos programables en PL/SQL:

- 2 Procedimientos almacenados: `sp_registrar_cita` y `sp_actualizar_estado_cita`.
- 2 Funciones: `fn_citas_pendientes_mascota` y `fn_disponibilidad_veterinario`.
- 2 Triggers: `trg_valida_fecha_cita` y `trg_auditoria_estado_cita`.
- 1 Package (`pkg_gestion_citas`) que agrupa 2 procedimientos adicionales (`reprogramar_cita`, `cancelar_cita`) y 2 funciones adicionales (`total_citas_dia`, `citas_por_veterinario`).

3. Modelo de datos utilizado

Se crearon 5 tablas en Oracle para soportar el módulo: `clientes`, `veterinarios`, `mascotas`, `citas` y `auditoria_citas` (esta última utilizada por el trigger de auditoría). El script completo de creación se encuentra en el archivo `vetcare_citas_oracle.sql` adjunto a esta entrega.

4. Procedimientos almacenados

4.1 `sp_registrar_cita`

Problema que resuelve: evita la doble reserva de un veterinario, validando que no exista ya una cita activa (pendiente o confirmada) para el mismo veterinario en la misma fecha y hora antes de insertar el registro. También valida que el veterinario exista y esté activo.

Código fuente

```
CREATE OR REPLACE PROCEDURE sp_registrar_cita (  
    p_id_mascota      IN  citas.id_mascota%TYPE,  
    p_id_veterinario IN  citas.id_veterinario%TYPE,  
    p_fecha_cita     IN  citas.fecha_cita%TYPE,  
    p_hora_cita      IN  citas.hora_cita%TYPE,  
    p_motivo         IN  citas.motivo%TYPE,  
    p_id_cita_out    OUT citas.id_cita%TYPE  
) IS  
    v_ocupado NUMBER;  
    v_vet_activo NUMBER;  
BEGIN  
    SELECT COUNT(*) INTO v_vet_activo FROM veterinarios  
    WHERE id_veterinario = p_id_veterinario AND activo = 1;  
  
    IF v_vet_activo = 0 THEN  
        RAISE_APPLICATION_ERROR(-20001, 'Veterinario no existe o inactivo.');    END IF;  
  
    SELECT COUNT(*) INTO v_ocupado FROM citas  
    WHERE id_veterinario = p_id_veterinario  
    AND fecha_cita = p_fecha_cita AND hora_cita = p_hora_cita
```

```

AND estado IN ('PENDIENTE', 'CONFIRMADA');

IF v_ocupado > 0 THEN
    RAISE_APPLICATION_ERROR(-20002, 'El veterinario ya tiene una cita en esa fecha/hora.');
```

END IF;

```

INSERT INTO citas (id_mascota, id_veterinario, fecha_cita, hora_cita, motivo, estado)
VALUES (p_id_mascota, p_id_veterinario, p_fecha_cita, p_hora_cita, p_motivo, 'PENDIENTE')
RETURNING id_cita INTO p_id_cita_out;

COMMIT;
END sp_registrar_cita;
```

Lógica implementada: el procedimiento primero valida reglas de negocio (veterinario activo, disponibilidad de horario) antes de ejecutar el INSERT, y devuelve el id de la nueva cita mediante un parámetro OUT, para que la aplicación pueda usarlo inmediatamente (por ejemplo, para mostrar un comprobante).

4.2 sp_actualizar_estado_cita

Problema que resuelve: implementa la máquina de estados de una cita, impidiendo modificar una cita que ya se encuentra en un estado final (ATENDIDA o CANCELADA).

Código fuente

```

CREATE OR REPLACE PROCEDURE sp_actualizar_estado_cita (
    p_id_cita      IN citas.id_cita%TYPE,
    p_nuevo_estado IN citas.estado%TYPE
) IS
    v_estado_actual citas.estado%TYPE;
BEGIN
    SELECT estado INTO v_estado_actual FROM citas WHERE id_cita = p_id_cita;

    IF v_estado_actual IN ('ATENDIDA', 'CANCELADA') THEN
        RAISE_APPLICATION_ERROR(-20003,
            'No se puede modificar una cita en estado final (' || v_estado_actual || ').');
```

END IF;

```

    UPDATE citas SET estado = p_nuevo_estado WHERE id_cita = p_id_cita;
    COMMIT;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20004, 'La cita indicada no existe.');
```

END sp_actualizar_estado_cita;

Lógica implementada: se consulta primero el estado actual de la cita; si es un estado final, se lanza un error controlado (RAISE_APPLICATION_ERROR) en lugar de permitir la actualización. Este cambio de estado, al modificar la columna estado, dispara automáticamente el trigger de auditoría (ver sección 6.2).

5. Funciones

5.1 fn_citas_pendientes_mascota

Problema que resuelve: entrega de forma inmediata cuántas citas pendientes tiene una mascota, información utilizada por el reporte "citas pendientes por mascota" y por alertas del panel administrativo.

Código fuente

```

CREATE OR REPLACE FUNCTION fn_citas_pendientes_mascota (
    p_id_mascota IN citas.id_mascota%TYPE
) RETURN NUMBER IS
    v_total NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_total FROM citas
    WHERE id_mascota = p_id_mascota AND estado = 'PENDIENTE';

    RETURN v_total;
END fn_citas_pendientes_mascota;
```

Lógica implementada: función escalar simple que retorna un NUMBER, reutilizable tanto desde PL/SQL como directamente en una sentencia SELECT (por ejemplo, en un reporte de mascotas con citas pendientes).

5.2 fn_disponibilidad_veterinario

Problema que resuelve: permite consultar, antes de agendar una cita, si un veterinario está disponible en una fecha y hora específicas, evitando que el usuario intente una reserva que `sp_registrar_cita` rechazará.

Código fuente

```
CREATE OR REPLACE FUNCTION fn_disponibilidad_veterinario (  
    p_id_veterinario IN citas.id_veterinario%TYPE,  
    p_fecha_cita      IN citas.fecha_cita%TYPE,  
    p_hora_cita       IN citas.hora_cita%TYPE  
) RETURN VARCHAR2 IS  
    v_ocupado NUMBER;  
BEGIN  
    SELECT COUNT(*) INTO v_ocupado FROM citas  
    WHERE id_veterinario = p_id_veterinario  
      AND fecha_cita = p_fecha_cita AND hora_cita = p_hora_cita  
      AND estado IN ('PENDIENTE', 'CONFIRMADA');  
  
    IF v_ocupado > 0 THEN RETURN 'OCUPADO';  
    ELSE RETURN 'DISPONIBLE'; END IF;  
END fn_disponibilidad_veterinario;
```

Lógica implementada: encapsula la misma regla de disponibilidad que usa `sp_registrar_cita`, mostrando cómo una función puede reutilizarse desde el procedimiento (o desde el package, ver sección 7) para mantener la lógica de negocio en un solo lugar.

6. Triggers

6.1 trg_valida_fecha_cita

Problema que resuelve: garantiza la integridad de los datos impidiendo registrar o reprogramar una cita con una fecha anterior a la fecha actual.

Código fuente

```
CREATE OR REPLACE TRIGGER trg_valida_fecha_cita  
BEFORE INSERT OR UPDATE OF fecha_cita ON citas  
FOR EACH ROW  
BEGIN  
    IF :NEW.fecha_cita < TRUNC(SYSDATE) THEN  
        RAISE_APPLICATION_ERROR(-20010, 'No se puede agendar una cita en una fecha pasada.');
```

Lógica implementada: trigger de tipo BEFORE que se dispara antes de insertar o actualizar la columna `fecha_cita`; compara la fecha propuesta (`:NEW.fecha_cita`) contra la fecha del servidor (`TRUNC(SYSDATE)`) y rechaza la operación completa si la fecha es pasada.

6.2 trg_auditoria_estado_cita

Problema que resuelve: mantiene un historial de auditoría de cada cambio de estado de una cita (estado anterior, estado nuevo, usuario de base de datos y fecha del cambio), sin depender de que la aplicación lo registre manualmente.

Código fuente

```
CREATE OR REPLACE TRIGGER trg_auditoria_estado_cita  
AFTER UPDATE OF estado ON citas  
FOR EACH ROW  
WHEN (OLD.estado != NEW.estado)  
BEGIN  
    INSERT INTO auditoria_citas (id_cita, estado_anterior, estado_nuevo, usuario_bd, fecha_cambio)  
    VALUES (:NEW.id_cita, :OLD.estado, :NEW.estado, USER, SYSDATE);  
END trg_auditoria_estado_cita;
```

Lógica implementada: trigger de tipo AFTER UPDATE que solo se ejecuta cuando el valor de la columna `estado` realmente cambia (cláusula WHEN), insertando una fila en `auditoria_citas` con el estado anterior y el nuevo. Esto se prueba en conjunto con `sp_actualizar_estado_cita` en la sección de pruebas.

7. Package: pkg_gestion_citas

El package agrupa procedimientos y funciones adicionales relacionados con el módulo de Citas, mostrando cómo Oracle permite organizar la lógica de negocio relacionada en una sola unidad reutilizable (specification + body).

7.1 Package Specification

```
CREATE OR REPLACE PACKAGE pkg_gestion_citas IS

    PROCEDURE reprogramar_cita (
        p_id_cita      IN citas.id_cita%TYPE,
        p_nueva_fecha  IN citas.fecha_cita%TYPE,
        p_nueva_hora   IN citas.hora_cita%TYPE
    );

    PROCEDURE cancelar_cita (
        p_id_cita      IN citas.id_cita%TYPE,
        p_motivo_cancelacion IN VARCHAR2
    );

    FUNCTION total_citas_dia (p_fecha IN DATE) RETURN NUMBER;

    FUNCTION citas_por_veterinario (
        p_id_veterinario IN citas.id_veterinario%TYPE
    ) RETURN SYS_REFCURSOR;

END pkg_gestion_citas;
```

7.2 Package Body (resumen)

reprogramar_cita: cambia la fecha/hora de una cita ya existente, reutilizando fn_disponibilidad_veterinario para validar que el nuevo horario esté libre antes de actualizar.

cancelar_cita: reutiliza sp_actualizar_estado_cita para pasar la cita a estado CANCELADA (lo que dispara la auditoría automáticamente) y además concatena el motivo de la cancelación al campo motivo.

total_citas_dia: función que retorna el número total de citas registradas para una fecha determinada, útil para el reporte "citas del día".

citas_por_veterinario: función que retorna un SYS_REFCURSOR con el detalle de todas las citas de un veterinario, permitiendo que la aplicación (o un bloque PL/SQL) recorra el resultado con FETCH, tal como se muestra en la Prueba 11 del script SQL.

```
PROCEDURE reprogramar_cita (...) IS
    v_id_veterinario citas.id_veterinario%TYPE;
    v_disponibilidad VARCHAR2(15);
BEGIN
    SELECT id_veterinario INTO v_id_veterinario FROM citas WHERE id_cita = p_id_cita;
    v_disponibilidad := fn_disponibilidad_veterinario(v_id_veterinario, p_nueva_fecha, p_nueva_hora);
    IF v_disponibilidad = 'OCUPADO' THEN
        RAISE_APPLICATION_ERROR(-20020, 'El veterinario ya tiene otra cita en ese horario.');
```

El código completo del package (specification y body) se encuentra en el archivo vetcare_citas_oracle.sql adjunto.

8. Ejecución de pruebas

El script SQL incluye 11 pruebas ejecutadas mediante bloques PL/SQL anónimos con DBMS_OUTPUT, que cubren el funcionamiento correcto y los casos de error de cada objeto programable:

- Prueba 1: registro exitoso de una cita con sp_registrar_cita.
- Prueba 2: intento de doble reserva — se espera el error -20002.
- Prueba 3: verificación de disponibilidad de un veterinario con fn_disponibilidad_veterinario.
- Prueba 4: actualización de estado con sp_actualizar_estado_cita.

- Prueba 5: verificación de que el trigger trg_auditoria_estado_cita registró el cambio.
- Prueba 6: conteo de citas pendientes de una mascota con fn_citas_pendientes_mascota.
- Prueba 7: intento de registrar una cita con fecha pasada — se espera el error -20010 (trigger trg_valida_fecha_cita).
- Prueba 8: reprogramación de una cita con pkg_gestion_citas.reprogramar_cita.
- Prueba 9: cancelación de una cita con pkg_gestion_citas.cancelar_cita.
- Prueba 10: total de citas de un día con pkg_gestion_citas.total_citas_dia.
- Prueba 11: recorrido del cursor de citas por veterinario con pkg_gestion_citas.citas_por_veterinario.

Capturas de pantalla Oracle

Confirmación terminada.

Prueba 1 OK -> Cita registrada con id: 1

Procedimiento PL/SQL terminado correctamente.

Prueba 2 OK (esperado) -> ORA-20002: El veterinario ya tiene una cita en esa fecha y hora.

Procedimiento PL/SQL terminado correctamente.

Prueba 3 -> Disponibilidad Dra. Fernandez 10:00: OCUPADO

Prueba 3 -> Disponibilidad Dra. Fernandez 11:00: DISPONIBLE

Procedimiento PL/SQL terminado correctamente.

Prueba 4 OK -> Estado de la cita 1 actualizado a CONFIRMADA.

Procedimiento PL/SQL terminado correctamente.

Prueba 5 OK -> Registros de auditoria para la cita 1: 1

Procedimiento PL/SQL terminado correctamente.

Prueba 6 -> Citas pendientes de Rocky: 0

Procedimiento PL/SQL terminado correctamente.

Prueba 7 OK (esperado) -> ORA-20010: No se puede agendar una cita en una fecha pasada.

ORA-06512: en "SYSTEM.TRG_VALIDA_FECHA_CITA", línea 3

ORA-04088: error durante la ejecución del disparador 'SYSTEM.TRG_VALIDA_FECHA_CITA'

Procedimiento PL/SQL terminado correctamente.

Prueba 8 OK -> Cita 1 reprogramada.

Procedimiento PL/SQL terminado correctamente.

Prueba 9 OK -> Cita 3 cancelada.

Procedimiento PL/SQL terminado correctamente.

Prueba 10 -> Total de citas para 07/07/2026: 1

Procedimiento PL/SQL terminado correctamente.

Prueba 11 -> Citas de la Dra. Fernandez:

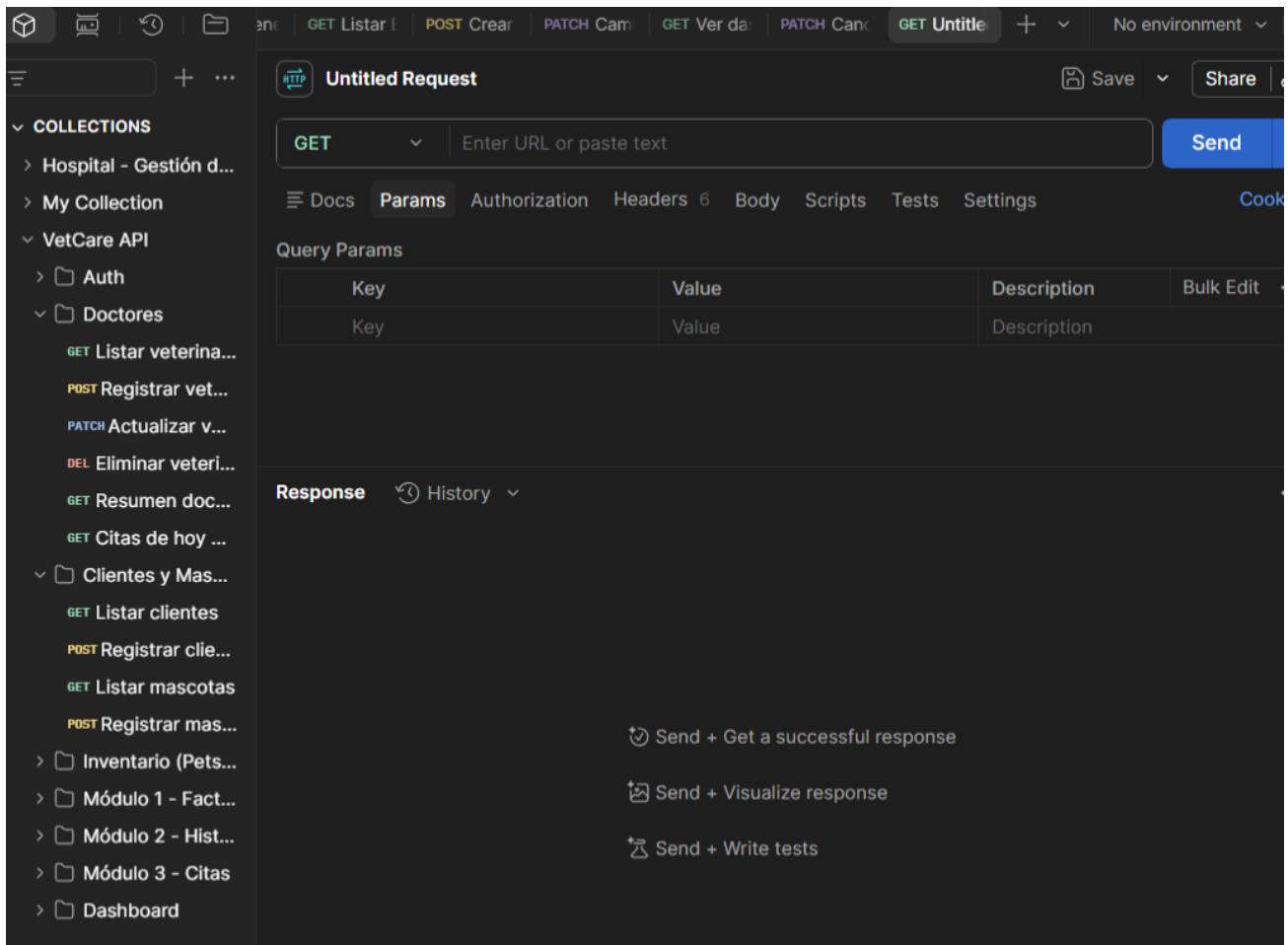
Cita #1 - Rocky - 07/07/2026 15:30 - PENDIENTE

Procedimiento PL/SQL terminado correctamente.

ID_CITA	ID_MASCOTA	ID_VETERINARIO	FECHA_CI	HORA	MOTIVO
1	1	1	07/07/26	15:30	Control anual
3	2	2	08/07/26	09:00	Desparasitacion Cancelada: El cliente reagendara la proxima semana

ID_AUDITORIA	ID_CITA	ESTADO_ANTERIOR	ESTADO_NUEVO	USUARIO_BD	FECHA_CA
1	1	PENDIENTE	CONFIRMADA	SYSTEM	05/07/26
2	1	CONFIRMADA	PENDIENTE	SYSTEM	05/07/26
3	3	PENDIENTE	CANCELADA	SYSTEM	05/07/26

Capturas de pantalla Postman



9. Conclusiones

- Los procedimientos almacenados permitieron centralizar en la base de datos las reglas de negocio críticas del módulo de Citas (evitar doble reserva, respetar la máquina de estados), reduciendo la posibilidad de que la aplicación cliente las omita o las implemente de forma inconsistente.
- Las funciones facilitaron exponer información reutilizable (disponibilidad, conteo de pendientes) tanto para otros objetos PL/SQL como para consultas directas en reportes.
- Los triggers demostraron ser una herramienta efectiva para garantizar integridad de datos (fechas válidas) y trazabilidad (auditoría de cambios de estado) de forma completamente transparente para la aplicación.
- El uso de un package permitió agrupar funcionalidades relacionadas (reprogramación, cancelación, reportes) bajo un mismo espacio de nombres, mejorando la organización y el mantenimiento del código PL/SQL del módulo.
- En conjunto, estos objetos programables mejoran la automatización, la consistencia y la seguridad de los datos del módulo de Citas de VetCare, más allá de lo que la capa de aplicación (Django) valida por sí sola.